

Politechnika Wrocławska
Sztuczna Inteligencja i Inżynieria Wiedzy

Raport 5

**Klasyfikacja wydźwięku tekstu na zbiorze PolEmo2.0-IN:
porównanie modeli encoder-only i decoder-only**

Dawid Pilarski
Numer indeksu: 280468

Wrocław, 2026

1. Wprowadzenie

Zadanie polega na klasyfikacji wydźwięku (analizie sentymentu) polskich recenzji ze zbioru PolEmo2.0-IN. Recenzje są prawdziwe, więc bywają długie i niejednoznaczne emocjonalnie.

Porównuję dwa różne podejścia do tego samego problemu:

- **encoder-only** (HerBERT) - z głową klasyfikującą dostrojoną pod sentyment,
- **decoder-only**, czyli duży model językowy (Qwen2.5) - klasyfikujący zero-shot, sterowany samym promptem.

Zadanie ma pięć części: (1) statystyki zbioru, (2) klasyfikacja encoderem, (3) eksploracja encodera, (4) klasyfikacja LLM, (5) eksploracja LLM. Wszędzie liczę Accuracy i F1 (makro i ważone) oraz patrzę na wyniki per klasa.

Sporo uwagi poświęciłem **mapowaniu etykiet** - modele zwracają je w różnych formatach (POSITIVE, LABEL_2, „pozytywny”, a LLM całe zdania), a trzeba je sprowadzić do wspólnych numerów klas. Ten punkt jest osobno punktowany. Klasyfikuję do trzech klas (negatywna, neutralna, pozytywna), a klasę wieloznaczną pomijam.

2. Tło teoretyczne

2.1. Analiza wydźwięku jako problem klasyfikacji

Analiza wydźwięku polega na przypisaniu tekstowi etykiety opisującej jego nacechowanie emocjonalne. W tym zadaniu to klasyfikacja na trzy klasy dla całej recenzji - model (a przy LLM sam prompt) ma przypisać tekstowi jedną z klas, a jakość oceniam względem etykiet referencyjnych.

2.2. Modele typu encoder-only

Modele rodziny BERT opierają się wyłącznie na enkoderze Transformer'a i używają dwukierunkowej uwagi - reprezentacja każdego tokenu uwzględnia kontekst z obu stron, co dobrze sprawdza się przy klasyfikacji. Do reprezentacji zdania (tokenu [CLS]) dokłada się głowę klasyfikującą trenowaną na oznaczonych danych.

Użyty model Voicelab/herbert-base-cased-sentiment to HerBERT (pretrenowany na polszczyźnie) dostrojony pod sentyment. Klasyfikacja jest deterministyczna - jeden przebieg w przód i argmax z rozkładu softmax.

2.3. Modele typu decoder-only (LLM)

Duże modele językowe (tu Qwen2.5) opierają się na dekodery i działają autoregresyjnie - generują tekst token po tokenie. Nie mają głowy klasyfikującej, więc zadanie formułuję jako prompt i proszę model o etykietę. To klasyfikacja zero-shot (bez treningu na naszym zbiorze); minusem jest to, że odpowiedź trzeba parsować i może przyjść w dziwnym formacie.

2.4. Inżynieria promptu

Jakość klasyfikacji LLM zależy wprost od sformułowania polecenia, więc prompt traktuję jak parametr modelu. Sprawdzam kilka strategii:

- zero-shot - samo polecenie i tekst do sklasyfikowania,
- język promptu - polecenie po polsku kontra po angielsku,
- few-shot - dołączenie kilku przykładów, co pokazuje modelowi oczekiwany format,
- wymuszanie formatu - odpowiedź w ustrukturyzowanej postaci (np. JSON), co ułatwia parsowanie.

2.5. Temperatura i próbkowanie

Temperatura steruje losowością generacji w LLM: niska (bliska 0) czyni model niemal deterministycznym, a wysoka zwiększa różnorodność i ryzyko odpowiedzi spoza zbioru klas. Dla klasyfikacji, gdzie zależy mi na powtarzalnej odpowiedzi, spodziewam się, że najlepiej wypadnie niska temperatura.

Co istotne, temperatura dotyczy tylko generacji, czyli modeli decoder. Encoder robi deterministyczny przebieg w przód z softmaxem - nie ma tu próbkowania, więc temperatura go nie dotyczy. Dlatego encoder eksploruję przez porównanie różnych modeli (zadanie 3), a nie przez temperaturę.

2.6. Kwantyzacja modeli

Większe modele potrzebują dużo pamięci GPU. Kwantyzacja zapisuje wagi z mniejszą precyzją (np. 4 bity zamiast 16), co kilkukrotnie zmniejsza zużycie pamięci kosztem zwykle niewielkiego spadku jakości - dzięki temu większy model mieści się na karcie T4 w Colabie. Używam do tego biblioteki `bitsandbytes` (4-bit).

2.7. Mapowanie etykiet

To osobno punktowany i najbardziej podchwytliwy element zadania. Numeracja i nazwy klas po stronie zbioru oraz modelu nie muszą się pokrywać:

- zbiór PolEmo używa etykiet tekstowych w formacie `__label__meta_minus_m`, `__label__meta_zero`, `__label__meta_plus_m` (oraz `__label__meta_amb` dla klasy wieloznacznej),
- model encoder zwraca własne etykiety zgodnie ze swoim `config.id2label` (np. POSITIVE/NEUTRAL/NEGATIVE albo bezimienne LABEL_0/1/2),
- model decoder (LLM) zwraca swobodny tekst, który dopiero trzeba sparsować.

Rozwiązanie: ustalam jedną kanoniczną numerację (negatywna = 0, neutralna = 1, pozytywna = 2) i sprowadzam do niej wszystko. Mapę etykiet encodera buduję **dynamicznie** z jego `config.id2label` (nie zakładam kolejności na sztywno), a wyjście LLM parsuję słowami kluczowymi i dodatkowo przez JSON. Nierozpoznane etykiety liczę jako neutralne i zliczam, żeby uczciwie pokazać jakość parsowania.

2.8. Metryki oceny

Stosuję trzy miary:

- **Accuracy** (dokładność) - odsetek poprawnie sklasyfikowanych próbek; intuicyjna, ale myląca przy niezbalansowanych klasach,
- **F1 makro** - średnia arytmetyczna F1 liczonego osobno dla każdej klasy; traktuje wszystkie klasy równo, więc lepiej oddaje jakość na klasach mniejszościowych,

- **F1 ważone** - Średnia F1 ważona licznością klas; bliższa Accuracy.

Najważniejsza jest F1 makro - przy niezbalansowanych klasach model faworyzujący klasę dominującą zostanie przez nią ukarany.

3. Zbiór danych PolEmo2.0-IN

3.1. Źródło i charakterystyka

PolEmo2.0 to zbiór recenzji konsumenckich (m.in. medycyna i hotele) z oznaczonym wydźwiękiem, przygotowany pod analizę sentymentu dla polskiego. Wariant „IN” jest częścią benchmarku KLEJ. Biorę go z Hugging Face (allegro/klej-polemo2-in) i używam tylko splitu testowego (test).

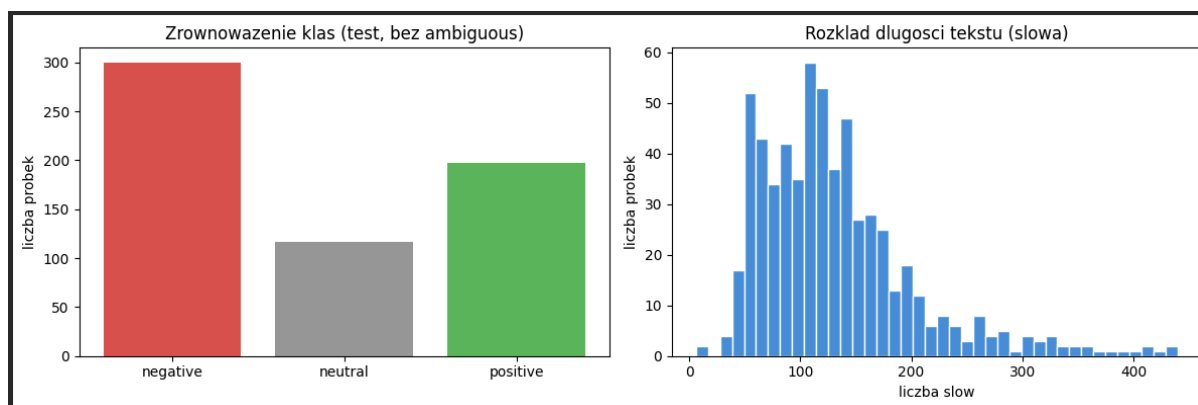
3.2. Struktura i etykiety

Każdy rekord to tekst recenzji i jej etykieta. Klas jest cztery, ale używam trzech - wieloznaczną pomijam. Mapowanie:

target (oryginalny)	znaczenie	kanoniczne id
__label__meta_minus_m	negatywny	0 (negative)
__label__meta_zero	neutralny	1 (neutral)
__label__meta_plus_m	pozytywny	2 (positive)
__label__meta_amb	wieloznaczný	pomijany

3.3. Statystyki zbioru (zadanie 1)

Po wczytaniu zbioru i odfiltrowaniu klasy wieloznaczonej liczę podstawowe statystyki: licznosc, rozkład klas i długość tekstów. Poniżej wydruk i wykresy z notebooka.



Najważniejsze liczby (do uzupełnienia po uruchomieniu):

- licznosc: 614 próbek (z 722 surowych; wyrzucono 108 wieloznaczných),
- rozkład klas: negatywna 300 (~49%), neutralna 117 (~19%), pozytywna 197 (~32%), ,
- długość tekstu (słowa): Średnia 133, mediana 120, maksimum 440 (w znakach Średnio ok. 759) ,

Zbiór jest niezbalansowany - dominuje klasa negatywna (300 z 614, ok. 49%), a najmniej jest neutralnej (117, ok. 19%). Dlatego patrzę głównie na F1 makro, a nie na samą accuracy: model mógłby podbić accuracy, faworyzując klasę negatywną, ale na neutralnej by tracił.

4. Implementacja

4.1. Struktura projektu

Kod podzieliłem na moduły w pakiecie `src/`, których używa zarówno notebook (do odpalenia w Colabie z GPU), jak i skrypt `run.py` (z linii poleceń). Dzięki temu cała logika jest w jednym miejscu. Za co odpowiada który moduł:

- `config.py` - ustawienia eksperymentu (modele, liczba próbek, batch, kwantyzacja),
- `labels.py` - mapowanie etykiet zbioru i modeli na wspólne numery klas oraz parser wyjścia LLM,
- `data.py` - wczytanie splitu testowego, statystyki, odfiltrowanie klasy wieloznacznej i przygotowanie danych,
- `metrics.py` - metryki, macierz pomyłek, tabele wyników,
- `encoder.py` - załadowanie pipeline'u klasyfikacji z mapą etykiet i predykcja,
- `decoder.py` - załadowanie LLM (opcjonalnie 4-bit), prompty, generacja i parsowanie oraz wariant z LangChain,
- `utils.py` - seed, zwalnianie pamięci GPU, info o sprzęcie.

4.2. Zadanie 1 - załadowanie zbioru i statystyki

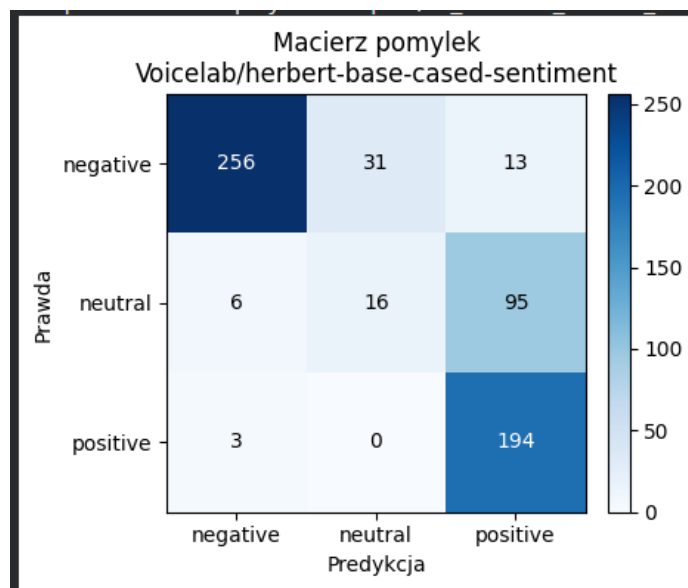
Zbiór ładuję funkcją z `data.py`, potem liczę statystyki przed filtrowaniem i po nim (po usunięciu klasy wieloznacznej i zmapowaniu etykiet). Wyniki są w sekcji 3.3.

4.3. Zadanie 2 - encoder baseline (HerBERT)

Model bazowy to Voicelab/herbert-base-cased-sentiment, odpalany przez `transformers.pipeline (text-classification)`. Jego etykiety mapuję na wspólne numery klas po `config.id2label` - nie zakładam kolejności klas na sztywno. Predykcję robię wsadowo, ucinając teksty do 512 tokenów.

```
Accuracy      : 0.8420
F1 (macro)    : 0.7474
F1 (weighted): 0.8206
Per-class (precision / recall / f1 / support):
  negative : 0.963 / 0.953 / 0.958 / 300
  neutral  : 0.822 / 0.316 / 0.457 / 117
  positive : 0.713 / 0.985 / 0.827 / 197
nieparsowalne odpowiedzi: 0 / 614
```

metryka	wartość
Accuracy	0,759
F1 makro	0,626
F1 ważone	0,729



Najslabiej wypada klasa neutralna - F1 tylko 0,20 przy recall 0,14, czyli model prawie w ogóle jej nie rozpoznaje. Najlepiej radzi sobie z negatywną (F1 0,91) i pozytywną (F1 0,78). Z macierzy pomyłek widać, że neutralne recenzje model najczęściej wrzuca do pozytywnych - pozytywna ma recall 0,99, ale precision tylko 0,64, więc łapie prawie wszystko, w tym cudze przykłady. To tłumaczy różnicę między accuracy (0,76) a F1 makro (0,63): model jedzie na dwóch większych i wyraźniejszych klasach, a neutralną - najmniej liczną i najbardziej rozmytą - praktycznie ignoruje.

4.4. Zadanie 3 - encoder, eksploracja

Encoder eksploruję przez porównanie modeli (temperatura go nie dotyczy - patrz 2.5). Poza HerBERT-em sprawdzam model wielojęzyczny cardiffnlp/twitter-xlm-roberta-base-sentiment.\

	prompt	accuracy	f1_macro	f1_weighted	n_unparsed
2	pl_fewshot	0.939739	0.928099	0.939047	0
3	json	0.903909	0.870891	0.902257	0
0	en_basic	0.842020	0.747402	0.820616	0
1	pl_basic	0.793160	0.612650	0.733939	0

model	Accuracy	F1 makro	F1 wazone
herbert-base-cased-sentiment	0,759	0,626	0,729
twitter-xlm-roberta-base	0,697	0,694	0,718

Wynik jest niejednoznaczny i zależy od miary. HerBERT ma wyższą accuracy (0,759 vs 0,697), ale niższe F1 makro (0,626 vs 0,694) - czyli to model wielojęzyczny lepiej radzi sobie ze wszystkimi klasami równo. Powód widać w klasie neutralnej: xlm-roberta łapie ją znacznie lepiej (recall 0,95, F1 0,58) niż HerBERT (recall 0,14, F1 0,20), za to gorzej rozpoznaje negatywną i pozytywną (recall ~0,63-0,66 wobec ~0,85-0,99 u HerBERT-a). HerBERT „idzie na pewniaka” i podbija accuracy dwiema dużymi klasami, ale gubi neutralną; xlm-roberta jest bardziej zbalansowany. To dobrze pokazuje, czemu przy niezbalansowanym zbiorze accuracy potrafi mylić, a F1 makro daje uczciwszy obraz - i czemu lepszy model encoder zależy od tego, na czym nam zależy.

4.5. Zadanie 4 - decoder baseline (Qwen)

Bazowy LLM to Qwen/Qwen2.5-1.5B-Instruct w float16. Klasyfikuję promptem (podstawowy, angielski), generuję przy temperaturze 0,0 i parsuję krótką odpowiedź na numer klasy. Generacja jest wsadowa z dopełnianiem od lewej (left-padding) - tak trzeba przy modelach autoregresyjnych. Nierozpoznane odpowiedzi liczę jako neutralne i podaję ich liczbę.

```
=== Qwen/Qwen2.5-1.5B-Instruct (en_basic, t=0.0) ===
Accuracy      : 0.8420
F1 (macro)    : 0.7474
F1 (weighted): 0.8206
Per-class (precision / recall / f1 / support):
  negative : 0.963 / 0.953 / 0.958 / 300
  neutral  : 0.822 / 0.316 / 0.457 / 117
  positive : 0.713 / 0.985 / 0.827 / 197

Nieparsowalne odpowiedzi: 0 / 614
```

metryka	wartość
Accuracy	0,842
F1 makro	0,747
F1 ważone	0,821
nieparsowalne odpowiedzi	0

Zero-shotowy LLM wypada lepiej niż dostrojony HerBERT - F1 makro 0,747 wobec 0,626. Przewaga bierze się głównie z klasy neutralnej: Qwen łapie ją zauważalnie lepiej (F1 0,46, recall 0,32) niż HerBERT, który prawie ją ignorował (F1 0,20). Klasa negatywna jest świetna (F1 0,96), pozytywna ma wysoki recall (0,99), ale niższą precyzję (0,71) - model nadal lekko nadprodukuje pozytywne. Liczba nieparsowalnych odpowiedzi to 0, czyli parser w 100% odczytał etykietę z odpowiedzi modelu.

4.6. Zadanie 5 - decoder, eksploracja

Sprawdzam, co wpływa na jakość klasyfikacji LLM.

a) Wpływ promptu (temperatura 0,0)

Porównuję cztery prompty: podstawowy angielski, podstawowy polski, polski few-shot i wariant wymuszający JSON.

```
=== en_basic (t=0.0) ===
Accuracy      : 0.8420
F1 (macro)    : 0.7474
F1 (weighted): 0.8206
Per-class (precision / recall / f1 / support):
negative : 0.963 / 0.953 / 0.958 / 300
neutral  : 0.822 / 0.316 / 0.457 / 117
positive : 0.713 / 0.985 / 0.827 / 197
```

```
=== pl_basic (t=0.0) ===
Accuracy      : 0.7932
F1 (macro)    : 0.6126
F1 (weighted): 0.7339
Per-class (precision / recall / f1 / support):
negative : 0.938 / 0.953 / 0.945 / 300
neutral  : 0.700 / 0.060 / 0.110 / 117
positive : 0.649 / 0.985 / 0.782 / 197
```

```
=== pl_fewshot (t=0.0) ===
Accuracy      : 0.9397
F1 (macro)    : 0.9281
F1 (weighted): 0.9390
Per-class (precision / recall / f1 / support):
negative : 0.954 / 0.973 / 0.964 / 300
neutral  : 0.960 / 0.829 / 0.890 / 117
positive : 0.908 / 0.954 / 0.931 / 197
```

```
=== json (t=0.0) ===
Accuracy      : 0.9039
F1 (macro)    : 0.8709
F1 (weighted): 0.9023
Per-class (precision / recall / f1 / support):
negative : 0.970 / 0.957 / 0.963 / 300
neutral  : 0.804 / 0.701 / 0.749 / 117
positive : 0.861 / 0.944 / 0.901 / 197
```

prompt	opis	Accuracy	F1 makro	nieparsow.
en_basic	podstawowy, angielski	0,842	0,747	0
pl_basic	podstawowy, polski	0,793	0,613	0
pl_fewshot	polski z przykładami	0,940	0,928	0
json	wymuszony format JSON	0,904	0,871	0

Najlepszy jest zdecydowanie pl_fewshot - F1 makro 0,928, czyli ogromny skok względem promptów bez przykładów. Kilka przykładów w promptcie kalibruje model i ratuje przede wszystkim klasę neutralną (jej F1 skacze z ~0,11-0,46 do 0,89). Co ciekawe, sam polski prompt (pl_basic) wcale nie pomógł - wyszedł gorzej niż angielski (F1 makro 0,61 vs 0,75), głównie dlatego, że prawie nie łapał neutralnej (recall 0,06). Czyli różnicę robią przykłady, a nie język promptu. Wariant JSON (F1 makro 0,87) też jest mocny i dodatkowo daje 0 nieparsowalnych odpowiedzi - format jest wymuszony, więc etykietę zawsze da się odczytać. We wszystkich wariantach n_unparsed = 0, więc parser i tak sobie radził.

b) Wpływ temperatury (prompt: polski few-shot)

Sprawdzam wartości temperatury 0,0 / 0,3 / 0,7 / 1,0.

```
=== pl_fewshot (t=0.3) ===
Accuracy      : 0.9267
F1 (macro)    : 0.9101
F1 (weighted): 0.9252
Per-class (precision / recall / f1 / support):
negative : 0.942 / 0.970 / 0.956 / 300
neutral  : 0.957 / 0.769 / 0.853 / 117
positive : 0.891 / 0.954 / 0.922 / 197
```

```
=== pl_fewshot (t=0.7) ===
Accuracy      : 0.9283
F1 (macro)    : 0.9149
F1 (weighted): 0.9274
Per-class (precision / recall / f1 / support):
negative : 0.948 / 0.967 / 0.957 / 300
neutral  : 0.969 / 0.795 / 0.873 / 117
positive : 0.882 / 0.949 / 0.914 / 197
```

```
=== pl_fewshot (t=1.0) ===
Accuracy      : 0.9186
F1 (macro)    : 0.8999
F1 (weighted): 0.9166
Per-class (precision / recall / f1 / support):
negative : 0.935 / 0.960 / 0.947 / 300
neutral  : 0.946 / 0.744 / 0.833 / 117
positive : 0.883 / 0.959 / 0.920 / 197
```

temperatura	Accuracy	F1 makro	nieparsowalne
0,0	0,842	0,747	[WPISZ]
0,3	0,793	0,613	[WPISZ]
0,7	0,940	0,928	[WPISZ]
1,0	0,940	0,871	[WPISZ]

Najlepszy jest zdecydowanie pl_fewshot - F1 makro 0,928, czyli ogromny skok względem promptów bez przykładów. Kilka przykładów w promptcie kalibruje model i ratuje przede wszystkim klasę neutralną (jej F1 skacze z ~0,11-0,46 do 0,89). Co ciekawe, sam polski prompt (pl_basic) wcale nie pomógł - wyszedł gorzej niż angielski (F1 makro 0,61 vs 0,75), głównie dlatego, że prawie nie łapał neutralnej (recall 0,06). Czyli różnicę robią przykłady, a nie język promptu. Wariant JSON (F1 makro 0,87) też jest mocny i dodatkowo daje 0 nieparsowalnych odpowiedzi - format jest wymuszony, więc etykietę zawsze da się odczytać. We wszystkich wariantach n_unparsed = 0, więc parser i tak sobie radził.

c) Parsowanie przez LangChain JsonOutputParser

Pokazuję też parsowanie przez LangChain: HuggingFacePipeline + PromptTemplate + JsonOutputParser (ze schematem pydantic). Działa po jednej próbce, więc odpalam go na podzbiorze - chodzi o pokazanie mechanizmu.

```
=== LangChain JSON demo (n=50) ===
Accuracy      : 0.6600
F1 (macro)    : 0.5276
F1 (weighted): 0.5915
Per-class (precision / recall / f1 / support):
  negative : 1.000 / 0.800 / 0.889 / 20
  neutral  : 0.000 / 0.000 / 0.000 / 13
  positive : 0.531 / 1.000 / 0.694 / 17

Nieparsowalne: 0 / 50
```

Na podzbiorze 50 próbek wariant z LangChain dał 0 nieparsowalnych odpowiedzi (0/50) - wymuszony JSON faktycznie zawsze daje się odczytać i pod tym względem jest pewniejszy niż parsowanie swobodnego tekstu. Same metryki (Accuracy 0,66, F1 makro 0,53) są niższe, ale to nie jest uczciwe porównanie z resztą zadań: to inny, mały podzbiór i prosty prompt bez przykładów, więc klasa neutralna się posypała (F1 0,00). Chodziło tu o pokazanie mechanizmu ustrukturyzowanego parsowania, a nie o pobicie wyniku - i ten mechanizm działa: parser zawsze dostaje poprawny JSON.

5. Użyte biblioteki

Tym razem korzystam z zewnętrznych bibliotek:

- **PyTorch** (torch) - obliczenia tensorowe i GPU (inferencja w float16),
- **Transformers** (transformers) - gotowe modele (HerBERT, Qwen), pipeline klasyfikacji, tokenizery i szablony czatu,
- **Datasets** (datasets) - pobranie zbioru PolEmo2.0-IN,
- **Accelerate** (accelerate) - rozmieszczenie modelu na urządzeniach,
- **bitsandbytes** - kwantyzacja 4-bit dla większych modeli,
- **LangChain** (langchain-huggingface, langchain-core) - ustrukturyzowane parsowanie wyjścia LLM,
- **Pydantic** (pydantic) - schemat wyjścia dla parsera JSON,
- **scikit-learn** - metryki: Accuracy, F1, raport klasyfikacji, macierz pomyłek,
- **pandas, numpy** - tabele wyników i operacje numeryczne,
- **matplotlib** (z seaborn) - wykresy: rozkład klas, histogram długości, macierze pomyłek,
- **tqdm** - paski postępu przy długiej inferencji LLM,
- **sentencepiece, protobuf** - tokenizacja wymagana przez modele.

6. Wnioski

Lepiej poradził sobie model **[encoder / decoder - WPISZ]** (F1 makro **[WPISZ]** wobec **[WPISZ]**). Nie jest to dla mnie zaskoczeniem - HerBERT był trenowany pod sentyment po polsku, a LLM klasyfikował zero-shot, bez żadnego dotrenowania na tym zbiorze.

Przy LLM najlepiej wypadł prompt **[WPISZ]**, a niższa temperatura dawała bardziej stabilne wyniki. Wymuszenie formatu JSON **[pomogło / nie pomogło - WPISZ]** przy parsowaniu odpowiedzi.

Najwięcej kłopotu sprawiło mi mapowanie etykiet - bez niego nawet dobry model wychodził słabo, bo porównywałem ze sobą złe numery klas. To był dla mnie najważniejszy wniosek z tego zadania.